

BOCETO DE ENFOQUE

Agora como proyecto desarrollado con IA y tecnologías actuales

Propuesta para orientar la defensa hacia el proceso de desarrollo, integracion, despliegue y aprendizaje.

Indice

- Indice propuesto
- 1. Planteamiento del nuevo enfoque
- 2. Objetivo de la defensa
- 3. Problema funcional que resuelve Agora
- 4. Papel real de la inteligencia artificial
- 5. Fuentes de informacion utilizadas
- 6. Estructura general del sistema
- 7. Arquitectura por capas
- 8. Portal interno: tecnologia e implementacion
- 9. Portal externo: tecnologia e implementacion
- 10. Backend Symfony: API y reglas de negocio
- 11. Modelo de datos y persistencia
- 12. Conexion frontend-backend
- 13. Despliegue en Google Cloud
- 14. Docker, Docker Compose y contenedores
- 15. Proxy, HTTPS y URL publica
- 16. Correo transaccional con Brevo
- 17. Agora Desktop como supervision tecnica
- 18. Pruebas y validacion
- 19. Dificultades encontradas
- 20. Aprendizaje obtenido
- 21. Como se explicaria el proceso desde cero
- 22. Limitaciones y futuras mejoras
- 23. Propuesta de estructura para la futura presentacion
- 24. Cierre

Boceto de reorientacion de defensa: Agora como aplicacion funcional desarrollada con IA y tecnologías actuales

Indice propuesto

1. Planteamiento del nuevo enfoque
2. Objetivo de la defensa

3. Problema funcional que resuelve Agora
4. Papel real de la inteligencia artificial
5. Fuentes de informacion utilizadas
6. Estructura general del sistema
7. Arquitectura por capas
8. Portal interno: tecnologia e implementacion
9. Portal externo: tecnologia e implementacion
10. Backend Symfony: API y reglas de negocio
11. Modelo de datos y persistencia
12. Conexion frontend-backend
13. Despliegue en Google Cloud
14. Docker, Docker Compose y contenedores
15. Proxy, HTTPS y URL publica
16. Correo transaccional con Brevo
17. Agora Desktop como supervision tecnica
18. Pruebas y validacion
19. Dificultades encontradas
20. Aprendizaje obtenido
21. Como se explicaria el proceso desde cero
22. Limitaciones y futuras mejoras
23. Propuesta de estructura para la futura presentacion
24. Cierre

1. Planteamiento del nuevo enfoque

El proyecto Agora no se plantea como un proyecto nuevo ni como una reescritura del trabajo ya realizado. La aplicacion desarrollada se mantiene como base funcional. El cambio propuesto se centra en la forma de defenderlo.

El enfoque inicial podia interpretarse como una defensa de autoria manual completa del codigo. Ese enfoque no representa bien el proceso real de trabajo, porque durante el desarrollo se han utilizado herramientas actuales de apoyo, entre ellas inteligencia artificial generativa.

La nueva orientacion propone presentar Agora como un caso practico de desarrollo de una aplicacion funcional utilizando tecnologias actuales. Esto incluye:

- inteligencia artificial como apoyo al desarrollo;
- React para la interfaz;
- Symfony para el backend;
- Doctrine para la capa de datos;
- PostgreSQL para persistencia;
- Docker y Docker Compose para ejecucion en contenedores;
- Google Cloud para despliegue;
- Caddy para proxy y HTTPS;
- Brevo para correo;
- Electron para aplicacion de escritorio.

La idea principal de la defensa seria:

Agora demuestra como se puede pasar de una necesidad funcional a una aplicacion desplegada combinando IA generativa, tecnologias web, contenedores y cloud.

El objetivo no seria afirmar que todo el codigo se ha escrito manualmente linea por linea, sino explicar como se ha dirigido el proceso, como se han usado las herramientas y como se han validado los resultados.

2. Objetivo de la defensa

El objetivo de la nueva defensa es explicar el proyecto desde el proceso de construcción, integración, despliegue y aprendizaje.

La defensa debe demostrar:

- que se ha identificado una necesidad funcional concreta;
- que se ha planteado una estructura de aplicación completa;
- que se han utilizado tecnologías actuales de desarrollo;
- que se ha usado IA como herramienta de apoyo;
- que se ha desplegado la aplicación en cloud;
- que se han probado los flujos principales;
- que se han encontrado dificultades reales;
- que se han delimitado funcionalidades cerradas y mejoras futuras.

La defensa no se centraría solo en mostrar pantallas, sino en explicar cómo se ha llegado a esas pantallas y qué tecnologías intervienen en cada parte.

La idea es presentar el proyecto como un ejemplo de aprendizaje aplicado:

No solo se ha construido una aplicación, sino que se ha aprendido a integrar frontend, backend, base de datos, despliegue, correo, supervisión y pruebas con apoyo de IA.

Este enfoque permite justificar mejor el alcance del proyecto. En lugar de ocultar que se han usado herramientas de IA, se presenta ese uso como parte del contexto tecnológico actual.

3. Problema funcional que resuelve Agora

Agora esta orientado a la gestion de practicas entre un centro educativo y empresas colaboradoras.

El problema funcional se puede resumir en varios puntos:

- las empresas necesitan solicitar colaboracion con el centro;
- el centro necesita revisar y aprobar esas empresas;
- deben gestionarse convenios entre centro y empresa;
- los estudiantes deben asignarse a empresas concretas;
- deben existir tutores academicos y tutores profesionales;
- debe haber seguimiento de la practica;
- debe existir comunicacion entre empresa y centro;
- el sistema debe poder consultarse desde un entorno desplegado.

El flujo principal propuesto es:

1. La empresa entra al portal externo.
2. Crea una cuenta.
3. Rellena una solicitud de colaboracion.
4. Verifica su correo.
5. El centro revisa la solicitud desde el portal interno.
6. Si se aprueba, la empresa queda activa.
7. El centro crea un convenio.
8. Se crea una asignacion para un estudiante.
9. Se realiza seguimiento.
10. Se registra evaluacion o cierre.

Este flujo sirve como hilo conductor de toda la defensa.

4. Papel real de la inteligencia artificial

La inteligencia artificial se plantea como una herramienta de apoyo al desarrollo, no como sustitucion total del trabajo.

La IA se ha usado principalmente para:

- generar estructuras iniciales de codigo;
- proponer controladores backend;
- proponer componentes frontend;
- ayudar a interpretar errores;
- sugerir correcciones;
- generar ejemplos de validaciones;
- apoyar scripts de despliegue;
- ayudar a revisar flujos tecnicos.

La IA utilizada se puede describir como IA generativa conversacional y agente de apoyo a la programacion, principalmente mediante herramientas de OpenAI como ChatGPT/Codex.

El uso de IA se puede explicar asi:

La IA ha acelerado tareas de desarrollo y depuracion, pero el resultado ha tenido que ser probado, integrado y revisado.

El proyecto muestra tambien una limitacion importante:

- la IA puede generar codigo que parece correcto, pero que falla en un entorno real;
- puede mezclar sintaxis de SQLite y PostgreSQL;
- puede proponer soluciones incompletas;
- puede abrir demasiadas funcionalidades si no se controla el alcance.

Por eso el aprendizaje no esta solo en "usar IA", sino en usarla dentro de un proceso de validacion.

La documentacion final, la organizacion del enfoque y la preparacion de la defensa se presentan como trabajo propio de revision y explicacion del proyecto.

5. Fuentes de informacion utilizadas

La informacion tecnica del proyecto procede de dos bloques principales.

Fuentes internas del proyecto

La fuente principal es el propio codigo y configuracion del repositorio:

- `frontend/app` : portal interno;
- `frontend/company-portal` : portal externo;
- `backend/src` : backend Symfony;
- `backend/migrations` : migraciones de base de datos;
- `deploy/gcp` : despliegue en Google Cloud;
- `desktop` : Agora Desktop;
- `Dockerfile` ;
- `deploy/gcp/docker-compose.yml` ;
- scripts de arranque y validacion.

Documentacion oficial consultada

Para entender y justificar tecnologias se han usado referencias oficiales:

- React: <https://react.dev/>
- Symfony: <https://symfony.com/doc/current/>
- Doctrine ORM: <https://www.doctrine-project.org/projects/doctrine-orm/>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Docker Compose: <https://docs.docker.com/compose/>
- Google Compute Engine: <https://docs.cloud.google.com/compute/docs>
- Caddy reverse proxy: https://caddyserver.com/docs/caddyfile/directives/reverse_proxy
- Brevo transaccional/SMTP: <https://developers.brevo.com/docs/smtp-integration>
- Electron: <https://www.electronjs.org/docs/latest/>
- OpenAI Codex/IA de apoyo al codigo: <https://openai.com/codex/>

Estas fuentes sirven para explicar la funcion de cada tecnologia y contrastar el uso realizado en el proyecto.

6. Estructura general del sistema

Ahora se organiza como una aplicación con varias piezas conectadas.

Esquema general:

```
Usuario centro
  |
  v
Portal interno React (/app)
  |
  v
API Symfony (/api)
  |
  v
Doctrine ORM
  |
  v
PostgreSQL

Empresa externa
  |
  v
Portal externo React (/externo)
  |
  v
API Symfony portal empresa
  |
  v
PostgreSQL + correo Brevo
```

En cloud, estas piezas se ejecutan dentro de contenedores Docker:

```
Internet
  |
  v
Caddy proxy HTTPS
  |
  v
Contenedor app
  |-- Apache
  |-- PHP / Symfony
  |-- build React interno
  |-- build React externo
  |
  v
Contenedor PostgreSQL
```

Ahora Desktop queda como herramienta auxiliar:

Agora Desktop

```
|-- consulta URL cloud  
|-- revisa estado  
|-- accede por SSH  
|-- muestra logs  
|-- lanza validaciones
```

7. Arquitectura por capas

El sistema puede explicarse por capas para que sea mas facil entenderlo.

Capa de presentacion

Corresponde a los portales React:

- portal interno;
- portal externo.

Su funcion es mostrar pantallas, formularios, tablas, mensajes y estados.

Capa de comunicacion

Corresponde a la API HTTP.

El frontend llama a rutas como:

- `/api/empresas;`
- `/api/convenios;`
- `/api/asignaciones;`
- `/api/portal-company/overview.`

La informacion viaja en formato JSON.

Capa de negocio

Corresponde a Symfony.

Aqui se validan datos, se aplican reglas y se decide que operaciones son validas.

Capa de persistencia

Corresponde a Doctrine y PostgreSQL.

Doctrine trabaja con entidades PHP y PostgreSQL almacena los datos reales.

Capa de despliegue

Corresponde a Docker, Docker Compose, Caddy y Google Cloud.

Esta capa permite ejecutar la aplicacion fuera del entorno local.

8. Portal interno: tecnologia e implementacion

El portal interno esta desarrollado con React y se encuentra en `frontend/app`.

Su objetivo es servir como herramienta de gestion para el centro educativo.

Funcionalidades principales:

- listado y gestion de empresas;
- gestion de estudiantes;
- gestion de tutores academicos;
- gestion de tutores profesionales;
- gestion de convenios;
- gestion de asignaciones;
- mensajeria con empresas;
- acceso a documentacion;
- consulta de indicadores.

La comunicacion con el backend se centraliza en:

```
frontend/app/src/services/api.ts
```

Ese archivo actua como cliente de API. Su funcion es evitar que cada componente haga peticiones de forma aislada. Centraliza:

- URL base de la API;
- autentificacion;
- llamadas `GET`, `POST`, `PUT`, `DELETE`;
- gestion de errores;
- descarga de ficheros.

Ejemplo de flujo:

```
Pantalla de empresas
-> llama a api.ts
-> GET /api/empresas
-> Symfony devuelve JSON
-> React actualiza tabla
```

9. Portal externo: tecnologia e implementacion

El portal externo esta desarrollado con React y se encuentra en:

```
frontend/company-portal
```

Su objetivo es ofrecer una entrada controlada a empresas externas.

Funcionalidades principales:

- registro de cuenta de empresa;
- inicio de sesion;
- envio de solicitud de colaboracion;
- verificacion de correo;
- consulta del estado de la solicitud;
- panel de empresa;
- mensajeria con el centro;
- visualizacion de informacion asociada.

La diferencia principal con el portal interno es que el portal externo esta pensado para usuarios fuera del centro.

Flujo:

```
Empresa externa  
-> crea cuenta  
-> rellena solicitud  
-> verifica correo  
-> espera aprobacion  
-> accede al panel
```

Este portal consume endpoints especificos del backend:

- autenticacion de empresa;
- solicitud de colaboracion;
- estado de empresa;
- mensajes;
- documentos visibles para empresa.

10. Backend Symfony: API y reglas de negocio

Symfony se utiliza como backend principal.

El backend esta en:

```
backend/src
```

Los controladores principales estan en:

```
backend/src/Controller/Api
```

Controladores destacados:

- `EmpresaColaboradoraController.php`;
- `ConvenioController.php`;
- `AsignacionController.php`;
- `EmpresaSolicitudController.php`;
- `PortalCompanyController.php`;
- `MonitorController.php`.

Un controlador recibe una peticion HTTP, valida los datos, llama a servicios o repositorios y devuelve una respuesta JSON.

Ejemplo:

```
POST /api/asignaciones
-> AsignacionController
-> valida estudiante, empresa, convenio y tutores
-> crea AsignacionPractica
-> guarda con Doctrine
-> devuelve JSON
```

Symfony concentra reglas importantes:

- solo empresas activas pueden usarse en asignaciones;
- un estudiante no debe tener practicas activas incompatibles;
- algunos borrados solo los puede realizar admin;
- los datos deben validarse antes de guardarse.

11. Modelo de datos y persistencia

El modelo de datos se representa mediante entidades Doctrine.

Entidades principales:

- EmpresaColaboradora;
- EmpresaSolicitud;
- EmpresaPortalCuenta;
- Estudiante;
- TutorAcademico;
- TutorProfesional;
- Convenio;
- AsignacionPractica;
- Seguimiento;
- EvaluacionFinal;
- EmpresaMensaje.

Esquema simplificado:

```
EmpresaSolicitud
  -> puede convertirse en EmpresaColaboradora

EmpresaColaboradora
  -> tiene tutores profesionales
  -> tiene convenios
  -> tiene mensajes

Convenio
  -> pertenece a una empresa
  -> se usa en asignaciones

AsignacionPractica
  -> estudiante
  -> empresa
  -> convenio
  -> tutor academico
  -> tutor profesional opcional
  -> seguimientos
  -> evaluacion final
```

PostgreSQL almacena los datos en el despliegue cloud. Doctrine permite que Symfony trabaje con objetos PHP y los traduzca a operaciones SQL.

Las migraciones se ubican en:

backend/migrations

12. Conexion frontend-backend

La conexion entre frontend y backend es una de las partes clave que se debe explicar.

El frontend no se conecta directamente a PostgreSQL. La cadena es:

React

- > fetch HTTP
- > API Symfony
- > Doctrine
- > PostgreSQL

Ejemplo con empresas:

Usuario abre Empresas

- > React llama GET /api/empresas
- > Symfony recibe la peticion
- > Doctrine consulta EmpresaColaboradora
- > PostgreSQL devuelve filas
- > Symfony devuelve JSON
- > React pinta la tabla

Ejemplo con creacion de asignacion:

Formulario de asignacion

- > POST /api/asignaciones
- > Symfony valida datos
- > comprueba reglas
- > crea entidad AsignacionPractica
- > guarda en PostgreSQL
- > devuelve respuesta JSON

Este punto permite demostrar aprendizaje tecnico sin entrar a defender cada linea concreta del codigo.

13. Despliegue en Google Cloud

El despliegue se realiza sobre una maquina virtual de Google Cloud Compute Engine.

La VM permite ejecutar la aplicacion fuera del ordenador local.

Proceso general:

1. Crear VM.
2. Instalar Docker.
3. Subir o clonar el proyecto.
4. Configurar variables de entorno.
5. Construir imagen Docker.
6. Levantar servicios con Docker Compose.
7. Ejecutar migraciones.
8. Exponer la app por HTTP/HTTPS.
9. Validar portales y API.
10. Automatizar arranque.

La ventaja de este enfoque es que la aplicacion se puede probar desde una red externa y no solo desde localhost.

Comandos representativos:

```
docker compose --env-file deploy/gcp/.env.gcp -f deploy/gcp/docker-compose.yml up -d --build  
  
docker compose --env-file deploy/gcp/.env.gcp -f deploy/gcp/docker-compose.yml ps  
  
docker compose --env-file deploy/gcp/.env.gcp -f deploy/gcp/docker-compose.yml logs app
```

14. Docker, Docker Compose y contenedores

Docker se utiliza para empaquetar la aplicación y sus dependencias.

Docker Compose se utiliza para coordinar varios servicios.

Servicios principales:

```
app
  -> Apache
  -> PHP
  -> Symfony
  -> builds React

db
  -> PostgreSQL

proxy
  -> Caddy
  -> HTTP/HTTPS
```

El uso de contenedores aporta:

- entorno mas reproducible;
- separacion de servicios;
- despliegue mas ordenado;
- facilidad para reiniciar o reconstruir;
- control de volúmenes persistentes.

Los volúmenes son importantes porque permiten conservar datos aunque el contenedor se recree.

Ejemplo:

```
postgres_data
  -> datos de PostgreSQL

document_storage
  -> ficheros/documentos
```

15. Proxy, HTTPS y URL publica

Caddy se utiliza como proxy frontal.

Su funcion es recibir peticiones desde Internet y dirigirlas al contenedor correcto.

Esquema:

```
Navegador
-> HTTPS
-> Caddy
-> contenedor app
-> Symfony / React
```

La URL publica se ha resuelto con `nip.io`, que permite generar un dominio basado en la IP publica.

Ejemplo:

```
https://agora.IP_PUBLICA.nip.io
```

Esto permite acceder al proyecto sin comprar un dominio propio.

Problema detectado:

- al reiniciar la VM, la IP publica puede cambiar;
- si cambia la IP, tambien cambia la URL basada en `nip.io`.

Solucion planteada:

- lectura de IP actual;
- actualizacion de URL efectiva;
- soporte desde Agora Desktop para ver la URL cloud.

16. Correo transaccional con Brevo

Brevo se utiliza para el envío de correos transaccionales.

En Agora se usa principalmente en el portal externo:

- verificación de correo;
- notificaciones relacionadas con solicitudes.

Flujo:

```
Empresa se registra
-> backend genera token
-> backend prepara correo
-> Brevo envia email
-> empresa verifica enlace
```

El correo es importante porque diferencia un simple formulario de una experiencia más realista.

También introduce dificultades:

- variables de entorno;
- credenciales;
- URL pública correcta;
- evitar enlaces a localhost;
- controlar si el envío falla.

La integración con Brevo se plantea como ejemplo de uso de un servicio externo dentro de una aplicación web.

17. Agora Desktop como supervision tecnica

Agora Desktop es una aplicacion de escritorio creada con Electron.

Su objetivo no es sustituir los portales, sino apoyar la supervision tecnica.

Funciones:

- ver si el despliegue esta en modo local o cloud;
- consultar la URL efectiva;
- comprobar estado de servicios;
- leer logs;
- ejecutar validaciones;
- acceder al entorno remoto por SSH;
- comprobar contenedores.

Electron permite construir una aplicacion de escritorio con tecnologias web: JavaScript, HTML y CSS.

Esquema:

```
Agora Desktop
-> configuracion cloud
-> SSH a VM
-> docker ps / logs
-> comprobacion de URL
-> validacion de API
```

Esta parte ayuda a explicar que el proyecto no solo tiene portales, sino tambien una herramienta de operacion tecnica.

18. Pruebas y validacion

La validacion del proyecto se ha realizado en varios niveles.

Validacion manual

Se probaron flujos reales:

- acceso al portal interno;
- acceso al portal externo;
- registro de empresa;
- verificacion de correo;
- aprobacion de solicitud;
- creacion de convenio;
- creacion de asignacion;
- mensajeria;
- supervision desde Desktop.

Validacion tecnica

Se revisaron:

- respuestas HTTP 200;
- errores 401, 403, 422 y 500;
- logs de contenedores;
- estado de Docker;
- estado de `agora.service`;
- conexion con PostgreSQL;
- disponibilidad de URL publica.

Tests automatizados

Se prepararon pruebas para partes del backend y frontend.

El objetivo de estas pruebas es comprobar que cambios futuros no rompan flujos principales.

19. Dificultades encontradas

Durante el desarrollo aparecieron dificultades reales.

Migraciones SQLite/PostgreSQL

Algunas migraciones funcionaban en SQLite pero fallaban en PostgreSQL. Esto ocurrio con tipos o sintaxis como `AUTOINCREMENT` o `CLOB`.

Aprendizaje:

No todas las bases de datos aceptan la misma sintaxis. El entorno local y cloud deben estar alineados.

Permisos de Symfony

Aparecieron errores 500 porque Symfony no podia escribir cache en produccion.

Aprendizaje:

Una aplicacion puede arrancar, pero fallar al ejecutar por permisos del sistema de archivos.

URL publica

La IP publica de la VM podia cambiar.

Aprendizaje:

El despliegue necesita una estrategia para gestionar URL publica o dominio estable.

Documentos

La gestion documental resulto mas compleja porque implica metadatos y fichero fisico.

Aprendizaje:

Guardar un documento no es solo almacenar un nombre en base de datos; tambien hay que persistir el archivo real.

20. Aprendizaje obtenido

El aprendizaje principal es entender como se construye una aplicacion completa por capas y como cada tecnologia cubre una necesidad concreta dentro del sistema.

Aprendizaje sobre arquitectura

Una aplicacion web completa no se compone solo de pantallas. Tiene varias responsabilidades separadas:

- interfaz de usuario;
- comunicacion HTTP;
- logica de negocio;
- persistencia;
- despliegue;
- seguridad;
- observabilidad;
- validacion.

En Agora, esta separacion se concreta asi:

```
React
  -> muestra interfaz y recoge acciones

Symfony
  -> recibe peticiones y aplica reglas

Doctrine
  -> traduce entidades a operaciones de base de datos

PostgreSQL
  -> almacena datos persistentes

Docker
  -> empaqueta servicios

Google Cloud
  -> publica la aplicacion fuera del entorno local
```

Este aprendizaje es importante porque permite explicar que cada herramienta tiene una funcion y que el proyecto no depende de una sola tecnologia.

Aprendizaje sobre frontend

En el frontend se ha aprendido que React no guarda directamente los datos definitivos. React trabaja con estado de pantalla y con llamadas a la API.

Por ejemplo:

```
Usuario pulsa "Crear asignacion"
-> React recoge los campos del formulario
-> React envia un POST a la API
-> React espera respuesta
-> React muestra exito o error
```

Esto permite diferenciar entre:

- datos temporales del formulario;
- datos persistentes guardados en base de datos;
- mensajes de error mostrados al usuario;
- actualizacion visual tras recibir respuesta del backend.

Aprendizaje sobre backend

En el backend se ha aprendido que Symfony actua como punto central entre el frontend y la base de datos.

Un controlador no solo recibe datos. Tambien:

- valida que los campos sean correctos;
- comprueba permisos;
- aplica reglas de negocio;
- llama a repositorios o servicios;
- devuelve respuestas controladas.

Ejemplo:

```
POST /api/asignaciones
-> comprobar estudiante
-> comprobar empresa
-> comprobar convenio
-> comprobar tutores
-> comprobar solapamientos
-> guardar asignacion
```

Este aprendizaje permite explicar que una API no es solo una ruta, sino una capa donde se protege la coherencia del sistema.

Aprendizaje sobre base de datos

Tambien se ha aprendido que la base de datos no es simplemente un lugar donde guardar informacion. Define relaciones entre elementos.

Ejemplo:

Empresa

- > tiene convenios
- > tiene tutores profesionales
- > tiene mensajes

Convenio

- > pertenece a empresa
- > se usa en asignaciones

Asignacion

- > une estudiante, empresa, convenio y tutores

Este aprendizaje fue importante para entender por que no se pueden crear asignaciones sin comprobar antes empresa, convenio y estudiante.

Aprendizaje sobre despliegue

En despliegue se ha aprendido que una aplicacion que funciona en local no siempre funciona igual en cloud.

Problemas que ayudaron a aprender:

- diferencias entre SQLite y PostgreSQL;
- permisos de escritura en Symfony;
- variables de entorno;
- puertos expuestos;
- contenedores detenidos;
- IP publica cambiante;
- necesidad de logs.

Esto permite explicar que desplegar no es solo subir archivos, sino preparar un entorno completo.

Aprendizaje sobre IA

Tambien se ha aprendido que la IA generativa es util para acelerar el trabajo, pero necesita supervision.

La IA puede ayudar a:

- generar una primera version;
- explicar errores;
- proponer cambios;
- crear estructuras repetitivas;
- sugerir comandos;
- revisar codigo.

Pero no sustituye:

- probar el resultado;
- decidir el alcance;
- interpretar logs;
- comprobar seguridad;
- validar flujos funcionales;
- entender que se esta presentando.

La conclusion de este aprendizaje seria:

La IA ayuda a avanzar, pero el criterio y la validacion siguen siendo necesarios para convertir codigo generado en una aplicacion defendible.

Aprendizaje sobre gestion del alcance

Otro aprendizaje importante es que un proyecto puede crecer demasiado. En Agora se fueron incorporando muchas partes: portales, cloud, correo, documentos, chat, desktop, roles y pruebas.

Esto obliga a separar:

- flujo principal;
- funcionalidades secundarias;
- mejoras futuras.

Esta separacion permite defender mejor el proyecto porque no se intenta presentar todo como cerrado al 100 %, sino que se explica que partes estan operativas y que partes deberian evolucionar.

21. Como se explicaria el proceso desde cero

Para explicar el proceso a alguien que empieza desde cero, se podria plantear como una guia metodologica. La idea seria mostrar que no se empieza programando directamente, sino entendiendo el problema y dividiendo el sistema.

Paso 1: definir el problema

Lo primero seria definir que necesidad se quiere resolver.

En este caso:

Un centro educativo necesita gestionar empresas colaboradoras, convenios, estudiantes, tutores, asignaciones y comunicacion con empresas.

Este paso evita empezar por la tecnologia sin saber que flujo se quiere representar.

Paso 2: identificar usuarios

Despues se identifican los tipos de usuario:

- personal del centro;
- empresas externas;
- administrador tecnico.

Cada usuario necesita una experiencia distinta. Por eso se decide crear:

- portal interno para el centro;
- portal externo para empresas;
- Agora Desktop para supervision tecnica.

Paso 3: diseñar el flujo principal

Antes de crear pantallas se define el recorrido basico:

```
Empresa se registra
-> verifica correo
-> envia solicitud
-> centro aprueba
-> se crea convenio
-> se crea asignacion
-> se hace seguimiento
```

Este flujo actua como columna vertebral del proyecto.

Paso 4: definir datos y relaciones

Despues se identifican las entidades principales:

- empresa;
- solicitud;
- cuenta de empresa;
- estudiante;
- tutor academico;
- tutor profesional;
- convenio;
- asignacion;
- mensaje;
- seguimiento.

Tambien se definen relaciones:

```
Empresa -> Convenios
Empresa -> Tutores profesionales
Convenio -> Asignaciones
Asignacion -> Estudiante
Asignacion -> Tutor academico
Asignacion -> Seguimientos
```

Este paso permite construir despues la base de datos con sentido.

Paso 5: crear el backend

El backend se crea para exponer la API.

Primero se crean entidades y controladores. Despues se añaden validaciones y reglas.

Ejemplo:

```
/api/empresas
/api/convenios
/api/asignaciones
/api/portal-company/overview
```

El backend debe responder preguntas como:

- existe esta empresa?;
- el convenio esta vigente?;
- el estudiante ya tiene una practica activa?;
- el usuario tiene permisos?;
- faltan campos obligatorios?

Paso 6: crear el frontend

Con la API definida se crean las pantallas.

El portal interno necesita:

- tablas;
- formularios;
- modales;
- filtros;
- acciones.

El portal externo necesita:

- registro;
- login;
- solicitud;
- estado;
- panel de empresa;
- mensajes.

Paso 7: conectar frontend y backend

La conexion se hace mediante HTTP y JSON.

Ejemplo:

```
React envia:  
POST /api/empresas  
  
Symfony responde:  
{  
  "id": 1,  
  "nombre": "Empresa Demo",  
  "estadoColaboracion": "activa"  
}
```

React usa esa respuesta para actualizar la interfaz.

Paso 8: probar localmente

Antes de desplegar se prueban los flujos principales:

- login;
- crear empresa;
- crear convenio;
- crear asignacion;

- registrar empresa externa;
- enviar mensaje;
- comprobar errores.

Este paso ayuda a detectar fallos antes de llevarlo a cloud.

Paso 9: preparar Docker

Despues se prepara Docker para empaquetar la aplicacion.

La idea es que no dependa de instalar manualmente todo en la VM.

Se definen servicios:

```
app
db
proxy
```

Cada servicio tiene una responsabilidad.

Paso 10: desplegar en cloud

Una vez preparado Docker, se despliega en una VM.

Se comprueba:

- que los contenedores arranquen;
- que PostgreSQL este disponible;
- que Symfony ejecute migraciones;
- que la URL publica responda;
- que los portales carguen.

Paso 11: añadir servicios externos

Despues se integran servicios como Brevo para correo.

Aqui hay que cuidar que las URLs enviadas por correo no apunten a `localhost` , sino a la URL publica real.

Paso 12: añadir supervision

Finalmente se añade Agora Desktop como herramienta tecnica.

Su funcion es facilitar:

- lectura de estado;
- consulta de logs;
- validacion cloud;

- supervision de servicios.

Paso 13: revisar errores y limitar alcance

Durante todo el proceso se revisan errores y se decide que entra en la version final.

Si una funcionalidad no queda suficientemente robusta, se documenta como mejora futura.

Este planteamiento convierte el proyecto en una demostracion didactica:

Como usar herramientas actuales para construir una aplicacion funcional paso a paso.

La defensa puede apoyarse en esta idea para explicar el proyecto de forma ordenada: primero problema, despues arquitectura, despues implementacion, despues despliegue y finalmente aprendizaje.

22. Limitaciones y futuras mejoras

El proyecto tiene partes funcionales y partes que quedan como mejora futura.

Funcionalidades defendibles:

- portal interno;
- portal externo;
- registro y verificación;
- gestión de empresas;
- convenios;
- asignaciones;
- mensajería;
- despliegue cloud;
- supervisión Desktop.

Mejoras futuras:

- gestión documental más robusta;
- permisos avanzados por rol;
- multi-centro;
- dominio propio;
- backups automáticos;
- mayor cobertura E2E;
- auditoría avanzada;
- seguridad de producción.

La defensa debe presentar estas mejoras como delimitación realista del alcance.

23. Propuesta de estructura para la futura presentacion

Para una presentacion de unos 10 minutos, la estructura final deberia ser mas reducida que este boceto. La propuesta seria concentrarla en estos bloques:

1. Problema y cambio de enfoque.
2. Papel de la IA.
3. Arquitectura general.
4. Tecnologias por capa.
5. Despliegue cloud.
6. Dificultades reales.
7. Aprendizaje.
8. Mejoras futuras.
9. Conclusion.

El flujo entre portal interno y portal externo se explicaria solo como ejemplo dentro de la arquitectura, sin dedicarle demasiadas diapositivas. El objetivo seria priorizar el proceso seguido, las tecnologias utilizadas y el aprendizaje obtenido.

24. Cierre

El cierre de la defensa debería reforzar tres ideas.

Primera:

Agora es una aplicación funcional que representa un flujo real de gestión de prácticas.

Segunda:

El valor del proyecto está en haber integrado tecnologías actuales para construir y desplegar una solución completa.

Tercera:

La IA ha sido una herramienta de apoyo, pero el proceso ha requerido dirección, pruebas, revisión, corrección y aprendizaje.

La conclusión final podría formularse así:

Este proyecto muestra cómo se puede utilizar IA generativa junto con tecnologías web actuales para desarrollar una aplicación funcional, desplegarla en cloud y aprender el proceso técnico completo que hay detrás.